OPERATING SYSTEMS ORANGE ASSIGNMENT

PES-VCS (Version Control System)

NAME: Achinthya MB
SRN: PES1UG24CS019
SECTION: A

Objective:

Build a Git-like version control system using content-addressable storage, trees, index, and commits.

PHASE 1:

1a:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1b:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

PHASE 2:

2a :

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

2b:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ find .pes/objects -type f
.pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafafac66014e1585d775e
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ xxd .pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafafac66014e1585d775e
00000000: 7472 6565 2034 3900 3130 3036 3434 2064  tree 49.100644 d
00000010: 6562 7567 2e74 7874 00ab abab abab abab  ebug.txt........
00000020: abab abab abab abab abab abab abab abab  ................
00000030: abab abab abab abab ab                  .........
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

PHASE 3:

3a:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  deleted:    file1.txt
  deleted:    file2.txt

Untracked files:
  untracked:  tree.h
  untracked:  Makefile
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  commit.c
  untracked:  index.c
  untracked:  force_write.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  force_write
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  commit.h
  untracked:  pes.h

[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

3b:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ cat .pes/index
100644 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 1713620000 8 file1.txt
100644 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 1713620000 8 file2.txt
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

PHASE 4:

4a:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ ./pes log
commit b574aa301971f4f9d3f09bfd02ee62affdfca7ef8a09fd20ba44efb282da87cc
Author: PES User <pes@localhost>
Date:   1776694890

    Initial commit

[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

4b:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/b5/74aa301971f4f9d3f09bfd02ee62affdfca7ef8a09fd20ba44efb282da87cc
.pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafafac66014e1585d775e
.pes/refs/heads/main
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

4c:

```
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ cat .pes/refs/heads/main
b574aa301971f4f9d3f09bfd02ee62affdfca7ef8a09fd20ba44efb282da87cc
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
[04/20/26]seed@VM:~/PES1UG24CS019-pes-vcs$
```

PHASE 5:

Q5.1: Implementation of pes checkout <branch>

- Changes: Update .pes/HEAD to the new branch ref; overwrite .pes/index with the target tree.
- Action: Sync the working directory by adding, removing, or updating files to match the target commit.
- Complexity: Destructive filesystem operations and managing untracked files to avoid data loss.

Q5.2: Detecting a "Dirty Working Directory"

- Logic: Compare working file stat() metadata against the index.
- Verification: If metadata differs, re-hash the file. A conflict exists if the working hash index hash and the target branch version differs.

Q5.3: Detached HEAD State

- State: .pes/HEAD stores a raw commit hash instead of a branch name.
- Risk: Commits are orphaned if you switch away.
- Recovery: Find the last hash in pes log and manually create a new branch ref pointing to it.

PHASE 6:

Q6.1: Reachability Algorithm

- Algorithm: Mark-and-Sweep.
- Process: Start from all refs (roots). Traverse the graph and store "reachable" hashes in a Hash Set. Delete any object in .pes/objects/ not in that set.
- Visit Count: For 100k commits, you'd visit roughly 100k+ objects, depending on tree depth.

Q6.2: GC Race Conditions

- Danger: GC might delete a new blob created by pes add before pes commit links it to a ref.
- Solution: Git uses an mtime grace period (e.g., 2 weeks). It only prunes unreachable objects that haven't been modified recently.